Machine Translated by Google

CAPÍTULO 3

Procesos



Ceremonias

3.12 Describa las acciones que realiza un núcleo para cambiar de contexto entre pro-
impuestos.

3.13 Construya un árbol de procesos similar a la Figura 3.7. Para obtener información de
procesos para el sistema UNIX o Linux, utilice el comando ps -ael.
Utilice el comando `man ps` para obtener más información sobre el comando `ps` .
El administrador de tareas en los sistemas Windows no proporciona el ID del proceso
principal, pero la herramienta de monitorización de procesos , disponible en tech-
net.microsoft.com, ofrece una herramienta de árbol de procesos.

3.14 Explique la función del proceso init (o systemd) en UNIX y Linux.
sistemas en lo que respecta a la terminación del proceso.

3.15 Incluyendo el proceso padre inicial, ¿cuántos procesos se crean?
¿El programa que se muestra en la Figura 3.32?

```
#include <stdio.h> #include
<unistd.h>

int main() {

        entero i;

        para (i = 0; i < 4; i++) bifurcar();


        devolver 0;
}
```

Figura 3.1 ¿Cuántos procesos se crean?

Machine Translated by Google

3.16 Explique las circunstancias en las que se alcanzará la línea de código marcada como printf("LINE J") en la Figura 3.33.

3.17 Utilizando el programa de la Figura 3.34, identifique los valores de pid en las líneas A, B, C y D. (Suponga que los pid reales del proceso padre e hijo son 2600 y 2603, respectivamente).

3.18 Dé un ejemplo de una situación en la que las tuberías ordinarias sean más adecuadas que las tuberías con nombre y un ejemplo de una situación en la que las tuberías con nombre sean más adecuadas que las tuberías ordinarias.

3.19 Considere el mecanismo RPC . Describa las consecuencias indeseables que podrían surgir al no aplicar la semántica "como máximo una vez" ni "exactamente una vez" . Describa los posibles usos de un mecanismo que no ofrezca ninguna de estas garantías.

3.20 Utilizando el programa que se muestra en la Figura 3.35, explique cuál será la salida en las líneas X e Y.

```
#include <sys/types.h> #include
<stdio.h> #include <unistd.h>


int main() { pid t

pid;   –

        /* bifurcar un proceso hijo */ pid = fork();


        if (pid < 0) { /* ocurrió un error */ fprintf(stderr, "Error al crear
            el fork"); return 1;


        } else if (pid == 0) { /* proceso hijo */ execlp("/bin/
            ls","ls",NULL); printf("LÍNEA J");


        } else { /* proceso padre */ /* el padre
            esperará a que el hijo termine */ wait(NULL); printf("Hijo completado");


        }

        devolver 0;
}
```

Figura 3.2 ¿Cuándo se alcanzará la LÍNEA J ?

Machine Translated by Google


```
#include <sys/types.h> #include
<stdio.h> #include <unistd.h>


int main() { pid

t pid,-pid1;

        /* bifurcar un proceso hijo */ pid =
        fork();

        if (pid < 0) { /* ocurrió un error */ fprintf(stderr, "Error
            al crear el fork"); return 1;


        } else if (pid == 0) { /* proceso hijo */ pid1 = getpid();
            printf("hijo: pid =
            %d",pid); /* A */ printf("hijo: pid1 = %d",pid1); /* B */


        } else { /* proceso padre */ pid1 =
            getpid(); printf("padre:
            pid = %d",pid); /* C */ printf("padre: pid1 = %d",pid1); /* D
            */ wait(NULL);

        }

        devolver 0;
}
```

Figura 3.3 ¿Cuáles son los valores pid?

3.21 ¿Cuáles son las ventajas y desventajas de cada uno de los siguientes? Considere tanto el nivel del sistema como el nivel del programador. a. Comunicación síncrona y asíncrona b. Almacenamiento en búfer automático y explícito c. Envío por copia y envío por referencia d. Mensajes de tamaño fijo y de tamaño variable

Machine Translated by Google


```
#include <sys/types.h> #include
<stdio.h> #include <unistd.h>


#define TAMAÑO 5

entero nums[SIZE] = {0,1,2,3,4};

int main() { int i;

pid t pid;
      –

    pid = fork();

    si (pid == 0) { para (i = 0;
        i < SIZE; i++) {
            nums[i] *= -i; printf("HIJO:
            %d ",nums[i]); /* LÍNEA X */
        }
    }
    else if (pid > 0) { wait(NULL); for
        (i = 0; i < SIZE; i+
        +) printf("PARENT: %d ",nums[i]); /* LÍNEA Y
            */
    }

    devolver 0;
}
```

Figura 3.4 ¿Qué salida habrá en la línea X y en la línea Y?